

BOSONIZE-LEAN: Phase 1 Formalization Blueprint

Yellow Team Architect

October 4, 2026

1 Introduction and Architecture

Based on the Red Team Phase 1 critique, the monolithic proofs and trivial stubs (0, id, bottom) are strictly banned. This document strictly mirrors the required Lean 4 stubs in `Bosonize/Stubs/Phase1_Stubs.lean`.

2 1.1 Lattice and Index Types

We define the spatial lattice as a periodic ring and the momentum band as a centered interval.

```
def Lambda (L : Nat) [NeZero L] := ZMod L
def LambdaDual (L : Nat) := {n : Int // - (L : Int) < 2
  * n /\ 2 * n <= (L : Int)}
lemma card_LambdaDual (hL : Even L) : Fintype.card (
  LambdaDual L) = L := sorry
inductive Chirality | R | L
```

3 1.2 Umbral Calculus Core

The continuous derivatives are replaced entirely by finite differences on the integers.

```
def shiftE (f : Int -> R) : Int -> R := fun x => f (x +
  1)
```

```

def fwdDiff (f : Int -> R) : Int -> R := fun x => f (x +
  1) - f x
def bwdDiff (f : Int -> R) : Int -> R := fun x => f x -
  f (x - 1)

lemma discrete_leibniz (f g : Int -> R) (x : Int) :
  fwdDiff (f * g) x = fwdDiff f x * g x + shiftE f x *
    fwdDiff g x := sorry

lemma sum_fwdDiff_eq_bounds (f : Int -> R) (a b : Int) (
  hab : a <= b) :
  \sum x \in Ico a b, fwdDiff f x = f b - f a := sorry

lemma sum_by_parts_Lambda (f g : Lambda L -> R) :
  \sum x : Lambda L, fwdDiff f x * g x = - \sum x :
    Lambda L, shiftE f x * fwdDiff g x := sorry

def umbralBeta (f : Int -> R) : Int -> R := fun x => (x
  : R) * f (x - 1)
lemma umbral_heisenberg (f : Int -> R) (x : Int) :
  fwdDiff (umbralBeta f) x - umbralBeta (fwdDiff f) x =
    f x := sorry

```

4 1.3 Finite Fourier

Fourier modes on the periodic lattice.

```

def plane_wave (k : LambdaDual L) (x : Lambda L) :
  Complex :=
  \zeta ^ ((k : Int) * (x.val : Int))

lemma fwdDiff_plane_wave (hz : \zeta \neq 0) (k :
  LambdaDual L) (x : Lambda L) :
  fwdDiff (fun z => \zeta ^ ((k : Int) * z)) x.val = (\
    zeta ^ (k : Int) - 1) * plane_wave \zeta k x :=
  sorry

lemma plane_wave_ortho (hz : \zeta \neq 0) (k k' :
  LambdaDual L) :

```

```

\sum x : Lambda L, (plane_wave \zeta k x) * conj (
  plane_wave \zeta k' x) = if k = k' then (L :
    Complex) else 0 := sorry

```

5 1.4 CAR Representation

The explicit power-set implementation of the fermionic Fock space using the exact phase counting function.

```

structure CARRep (i : Type) (V : Type) where
  c : i -> Module.End Complex V
  cdag : i -> Module.End Complex V
  anticom_c_cdag : \forall i j, c i * cdag j + cdag j *
    c i = if i = j then 1 else 0
  anticom_c_c : \forall i j, c i * c j + c j * c i = 0

def FockSpace (i : Type) := Finset i -> Complex

def sign_func (i : i) (S : Finset i) : Complex :=
  (-1) ^ (S.filter (fun k => k < i)).card

def concrete_CAR : CARRep i (FockSpace i) := sorry
lemma dim_fock_space : Module.finrank Complex (FockSpace
  i) = 2 ^ Fintype.card i := sorry

```

6 1.5 Lattice AQFT Net

The local subalgebras. The Red Team flagged that $\perp$ was used previously. We strictly require Algebra.adjoin.

```

def Net (I : Finset (Lambda L)) : Subalgebra Complex (
  Module.End Complex (FockSpace (Chirality x Lambda L))
) :=
  Algebra.adjoin Complex (sorry)

lemma net_isotony (h : I \subteq J) : Net I \le Net J
  := sorry
lemma net_twisted_locality (h_disj : Disjoint I J) (A :
  Net I) (B : Net J) : sorry

```

7 1.6 & 1.7 Vacuum and Energy Budget

```
def vacuum_state : FockSpace (Chirality x LambdaDual L)
  := sorry
def energy_func (S : Finset (LambdaDual L)) : Int :=
  sorry
lemma energy_nonneg (S : Finset (LambdaDual L)) : 0 \le
  energy_func S := sorry
def BudgetSpace (K Nmax : Nat) : Submodule Complex (
  FockSpace (Chirality x LambdaDual L)) := sorry
```

8 1.8 Umbral Boson Fock Layer

```
def BosonFock := MvPolynomial (Fin Q) Complex

def a_dag (m : Fin Q) : Module.End Complex BosonFock :=
  LinearMap.mulLeft Complex (MvPolynomial.X m)

def a_ann (m : Fin Q) : Module.End Complex BosonFock :=
  MvPolynomial.pderiv m

lemma boson_ccr (m n : Fin Q) :
  a_ann m * a_dag n - a_dag n * a_ann m = if m = n then
    1 else 0 := sorry

lemma nilpotent_on_budget (K : Nat) (p : BosonFock) (hp
  : MvPolynomial.totalDegree p \le K) :
  (sorry : BosonFock) = 0 := sorry
```